

CMP5354
Software Design
UML Class Diagram Advanced
Relationships

+Notations: ArrayList<String>
+Example:HashMap<Class>

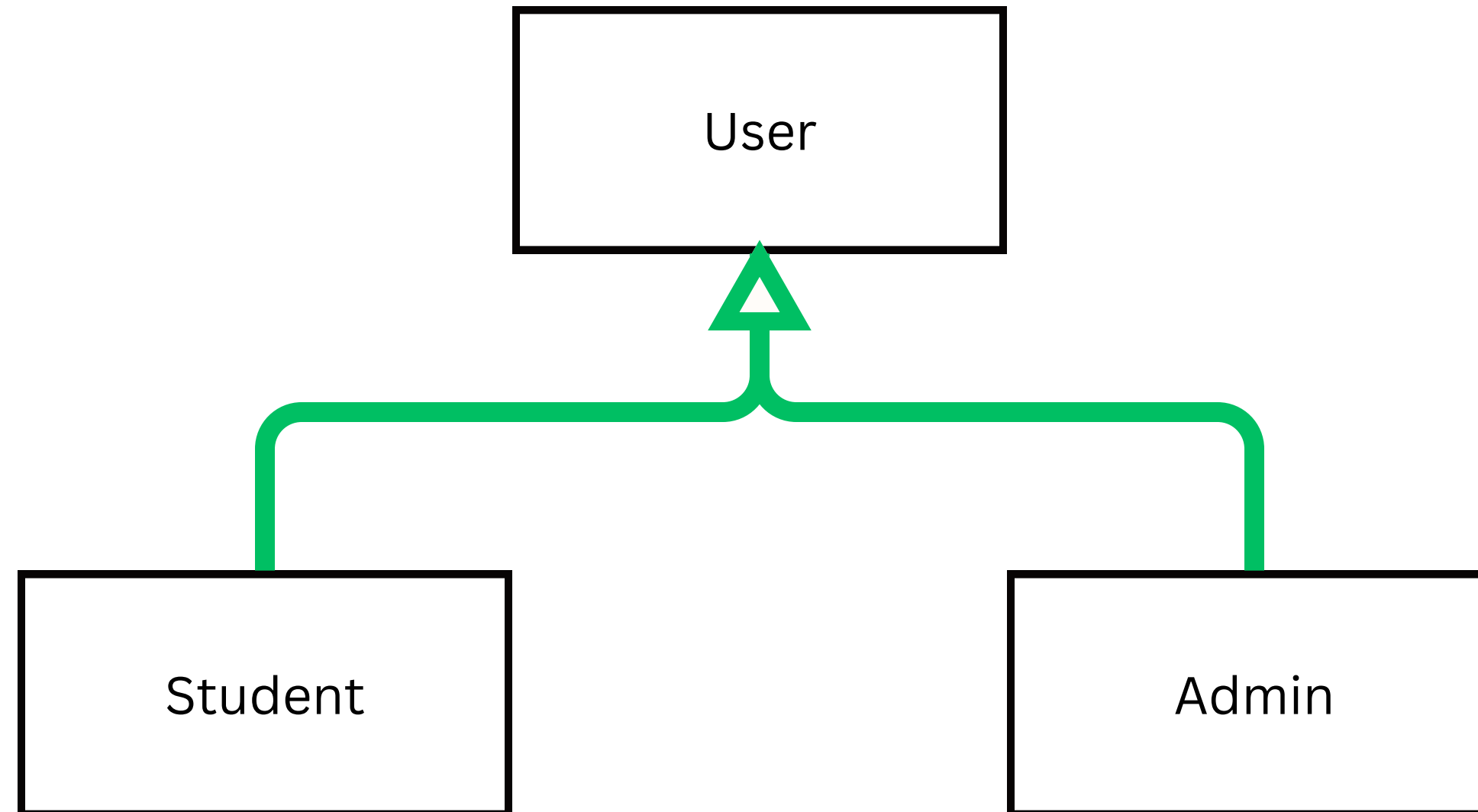
+setAuthor(String author)

UML Class Diagram Relationship Notations

Relationships

	Association	Any type of relationship or connection between classes
	Inheritance	A child class inherits functionality from the parent class.
	Aggregation	We want to convey that two classes are associated, but not as close as in direct association
	Composition	Relationships where the sub-object exists only as long as the container class exists

Inheritance



Refers to a type of relationship wherein one associated class is a child of another by virtue of assuming some attributes and methods of the parent class.

For example: both students and admins are users. The common attributes and methods go in the User class. The attributes and methods specific to the Student/Admin go in the sub-classes.

Inheritance -- Example from Computer Programming

```
class Vehicle:
    def __init__(self, make, model):
        self.__make = make
        self.__model = model
        self.__state = "stopped"

    def move(self):
        print("I am moving!")
        self.__state = "moving"

    def stop(self):
        print("I now stoppted.")
        self.__state = "stopped"

    def get_make(self):
        return self.__make

    def get_model(self):
        return self.__model

    def get_state(self):
        return self.__state

    def __str__(self):
        return f"{self.get_make()} {self.get_model()} is {self.get_state()}."
```

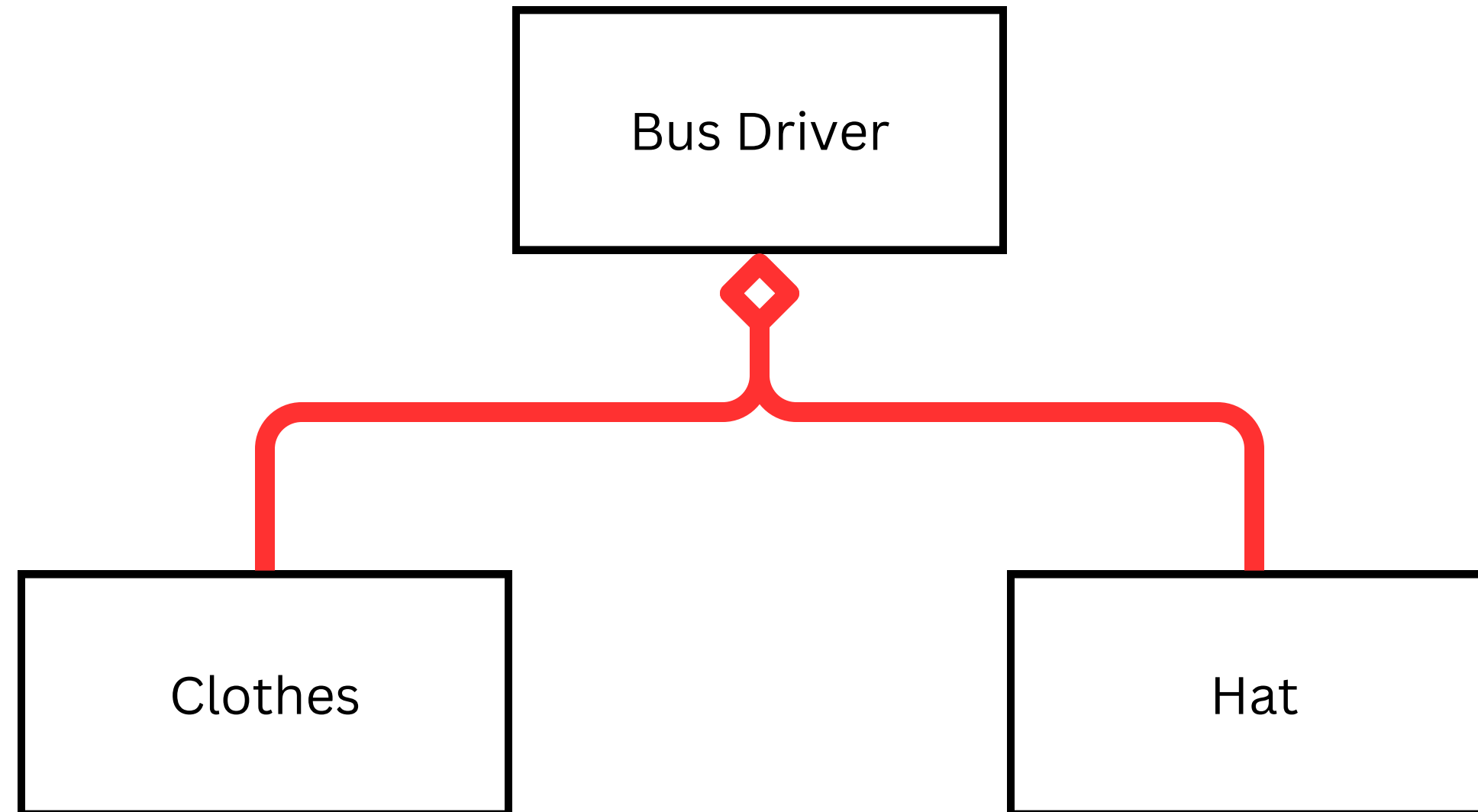
```
class Car(Vehicle):
    def __init__(self, make, model, doors):
        Vehicle.__init__(self, make, model)
        self.__doors_no = doors

    def set_doors_no(self, number):
        self.__doors_no = number

    def get_doors_no(self):
        return self.__doors_no

    def __str__(self):
        return f"{self.get_make()} {self.get_model()} with {self.get_doors_no()} doors is {self.get_state()}"
```


Aggregation



The contained classes are not strongly dependent on the lifecycle of the container

For example, bus drivers and work clothes and hats are part of the overall relationship, but they can be separated. Work clothes and hats can be worn on other drivers. Bus drivers can also wear other work clothes and hats.

Aggregation Example

```
class Clothes:
    def __init__(self, type_of_clothes):
        self.type_of_clothes = type_of_clothes

    def info(self):
        return f"Clothes: {self.type_of_clothes}"
```

Hat class

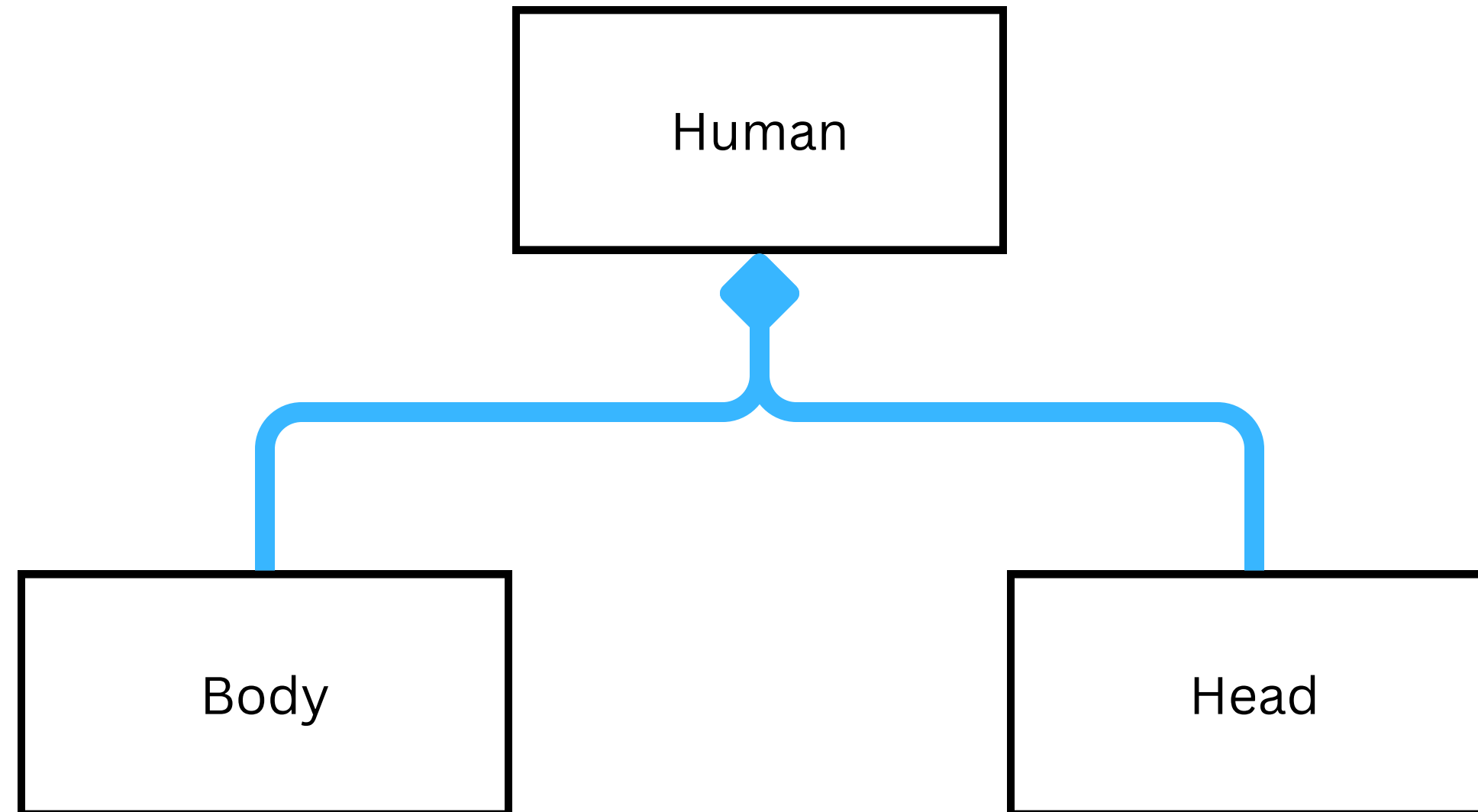
```
class Hat:
    def __init__(self, hat_type):
        self.hat_type = hat_type

    def info(self):
        return f"Hat type: {self.hat_type}"
```

Driver class (Aggregates Clothes and Hat)

```
class Driver:
    def __init__(self, name):
        self.name = name
        self.clothes = None # Aggregation: Clothes can be assigned externally
        self.hat = None # Aggregation: Hat can be assigned externally
```

Composition



The contained class will be obliterated when the container class is destroyed

For example, a person is composed of a head and a body. The two are inseparable and coexist

Composition Example

```
# Head class
class Head:
    def __init__(self, brain_size):
        self.brain_size = brain_size

    def info(self):
        return f"Head with brain size: {self.brain_size}"
```

```
# Body class
class Body:
    def __init__(self, height, weight):
        self.height = height
        self.weight = weight
```

```
    def info(self):
        return f"Body with height: {self.height} cm and weight: {self.weight} kg"
```

```
# Human class with strong composition (Human creates its Head and Body internally)
class Human:
    def __init__(self, name, brain_size, height, weight):
        self.name = name
        # Strong composition: Human is responsible for creating its Head and Body
        self.head = Head(brain_size) # Human creates the head internally
        self.body = Body(height, weight) # Human creates the body internally

    def human_info(self):
        return f"Human: {self.name}\n{self.head.info()}\n{self.body.info()}"
```

A stylized orange shopping cart icon. It features a rectangular body with rounded corners, a handle at the top, and two circular wheels at the bottom. The cart is positioned behind the text.

Online shopping cart example

User

- userID: string
- password: string
- loginStatus: string
- registerDate: date

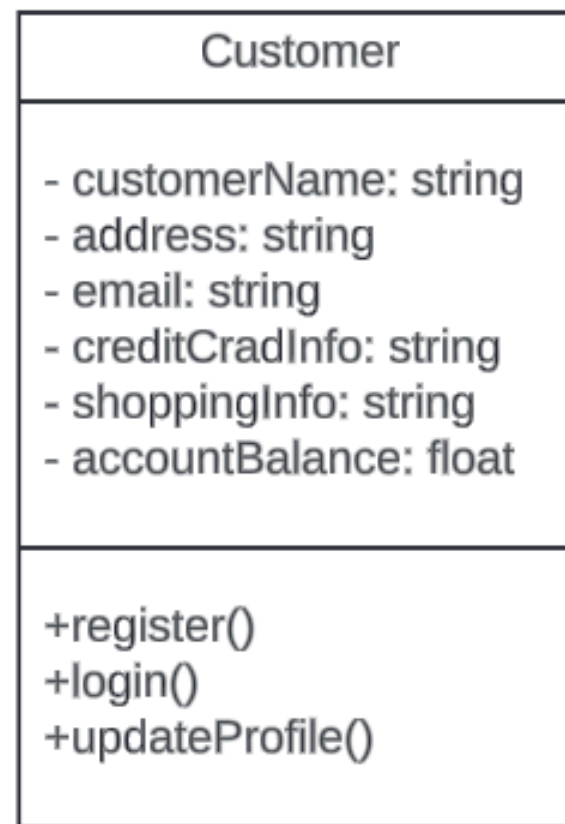
+verifyLogin(): bool

Customer
- customerName: string - address: string - email: string - creditCradInfo: string - shoppingInfo: string - accountBalance: float
+register() +login() +updateProfile()

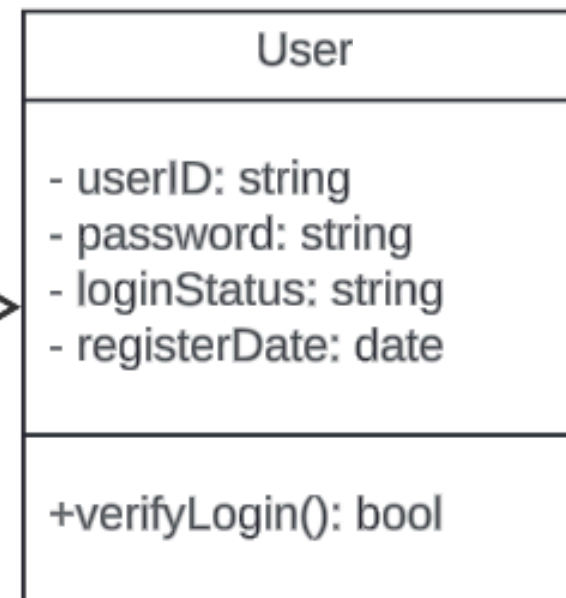
User
- userID: string - password: string - loginStatus: string - registerDate: date
+verifyLogin(): bool

Admin
- adminName: string - email: string
+updateCatalog(): bool

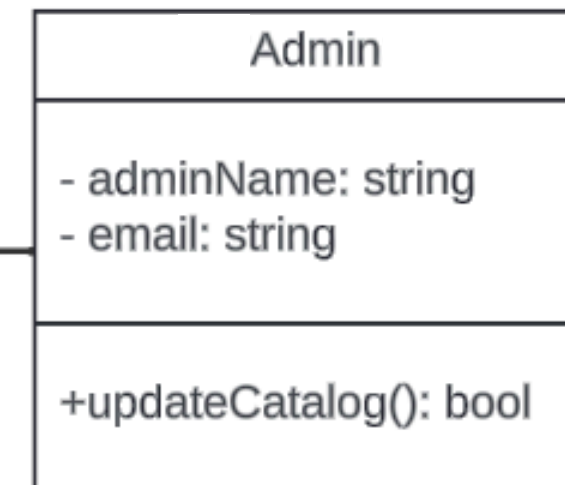
child



parent

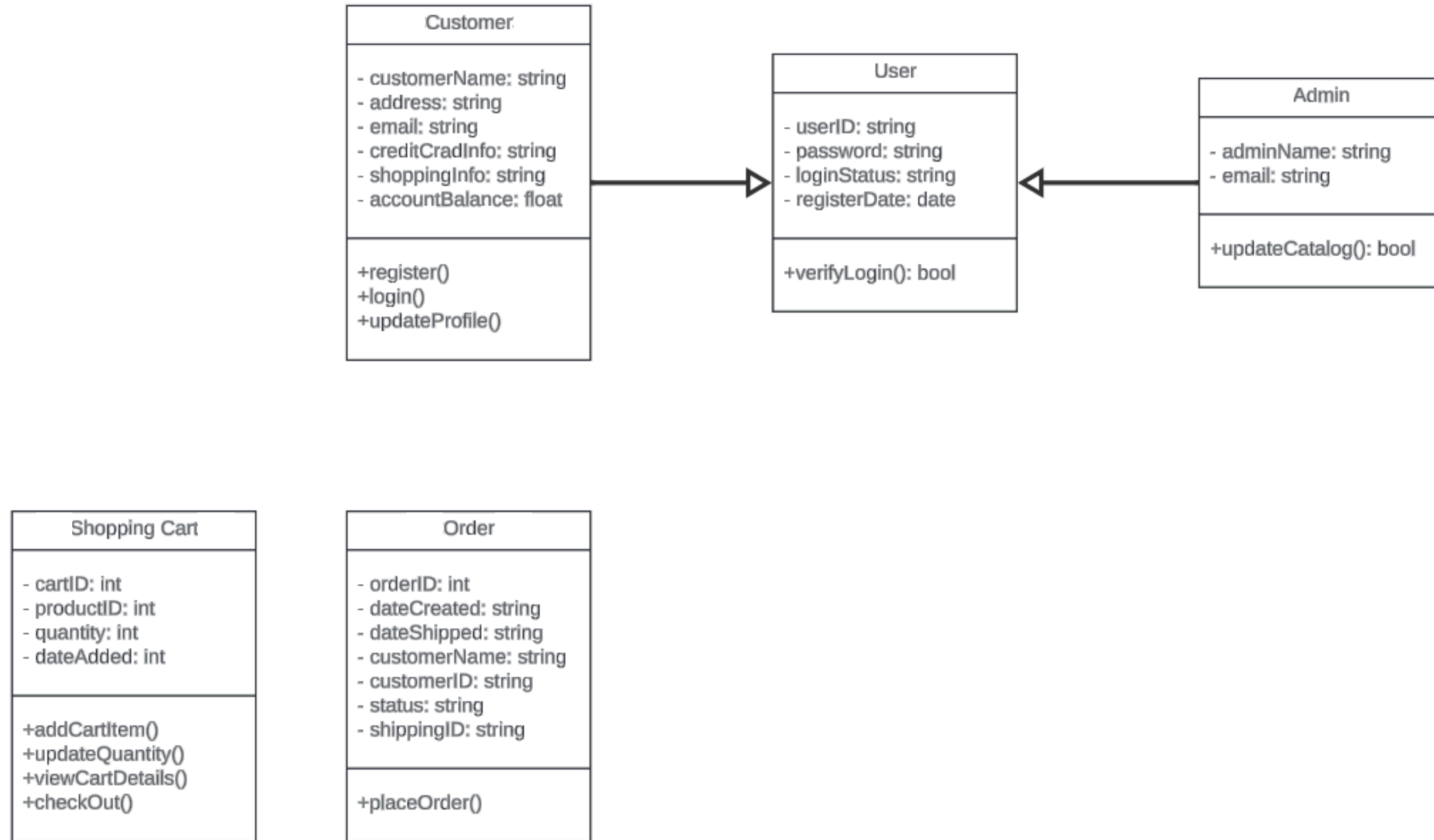


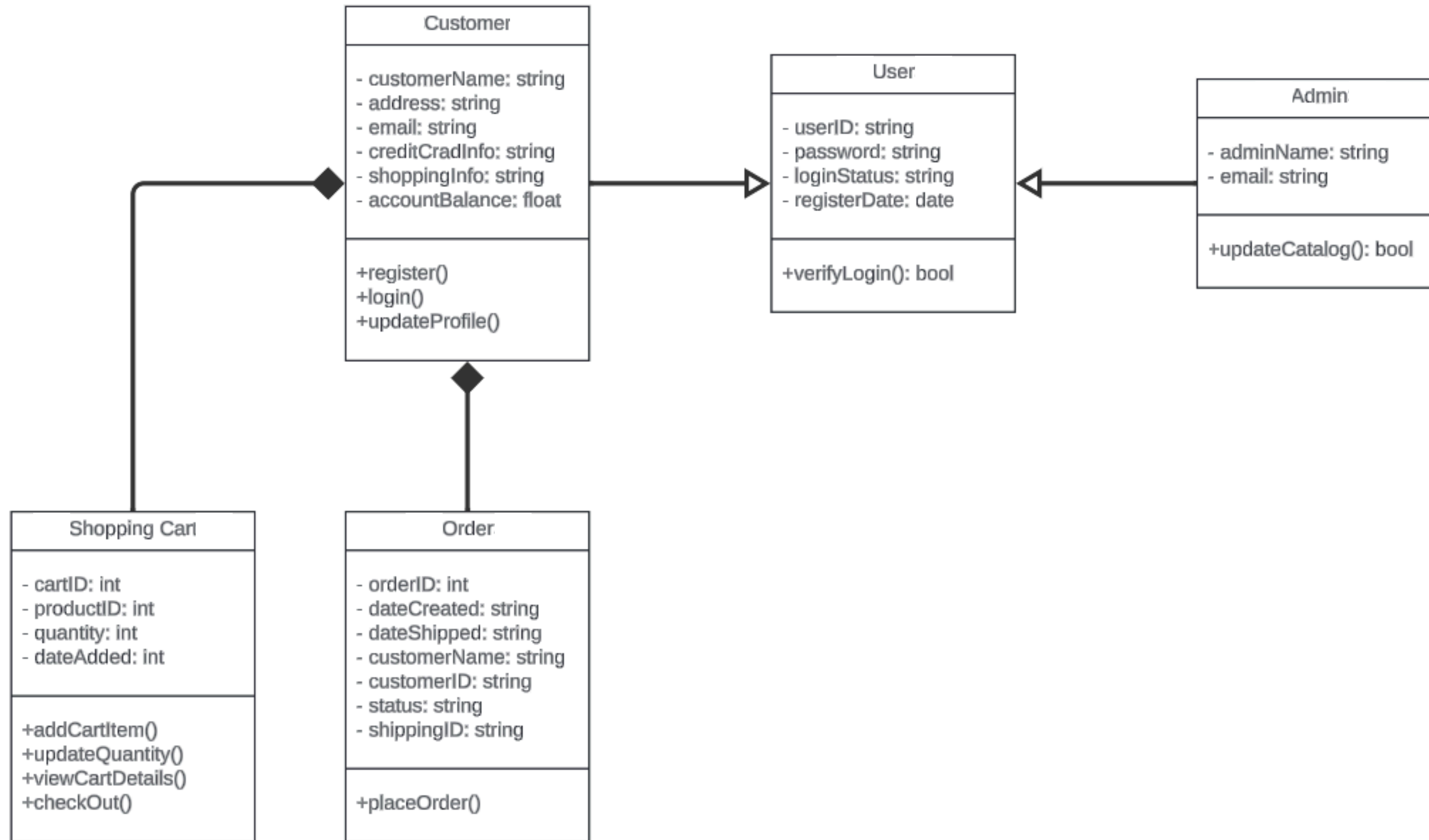
child



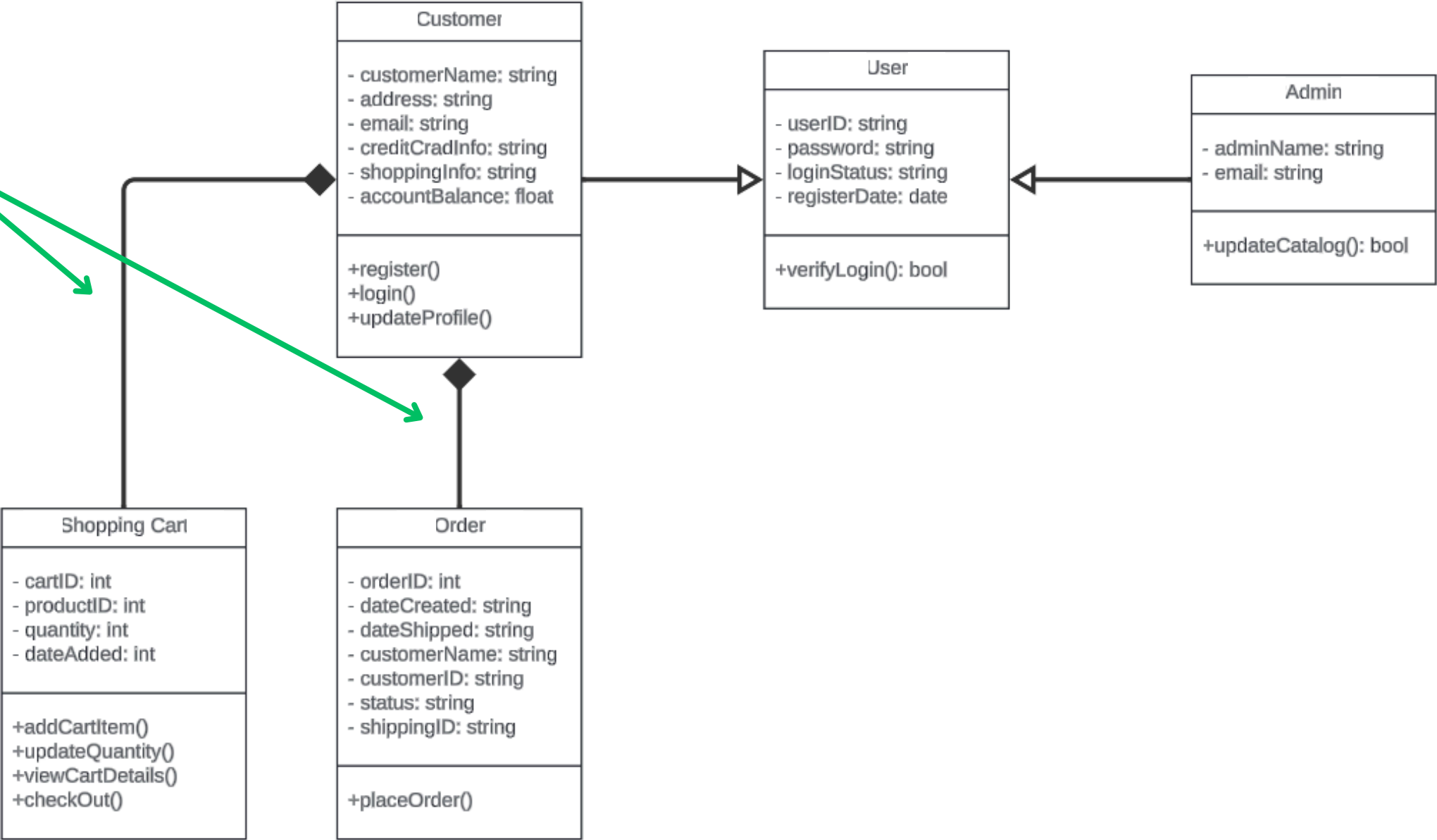
Tips for Inheritance:

- Only include child-specific methods and attributes in the subclasses.
- By default, the parent's methods and attributes are inherited by the child classes.

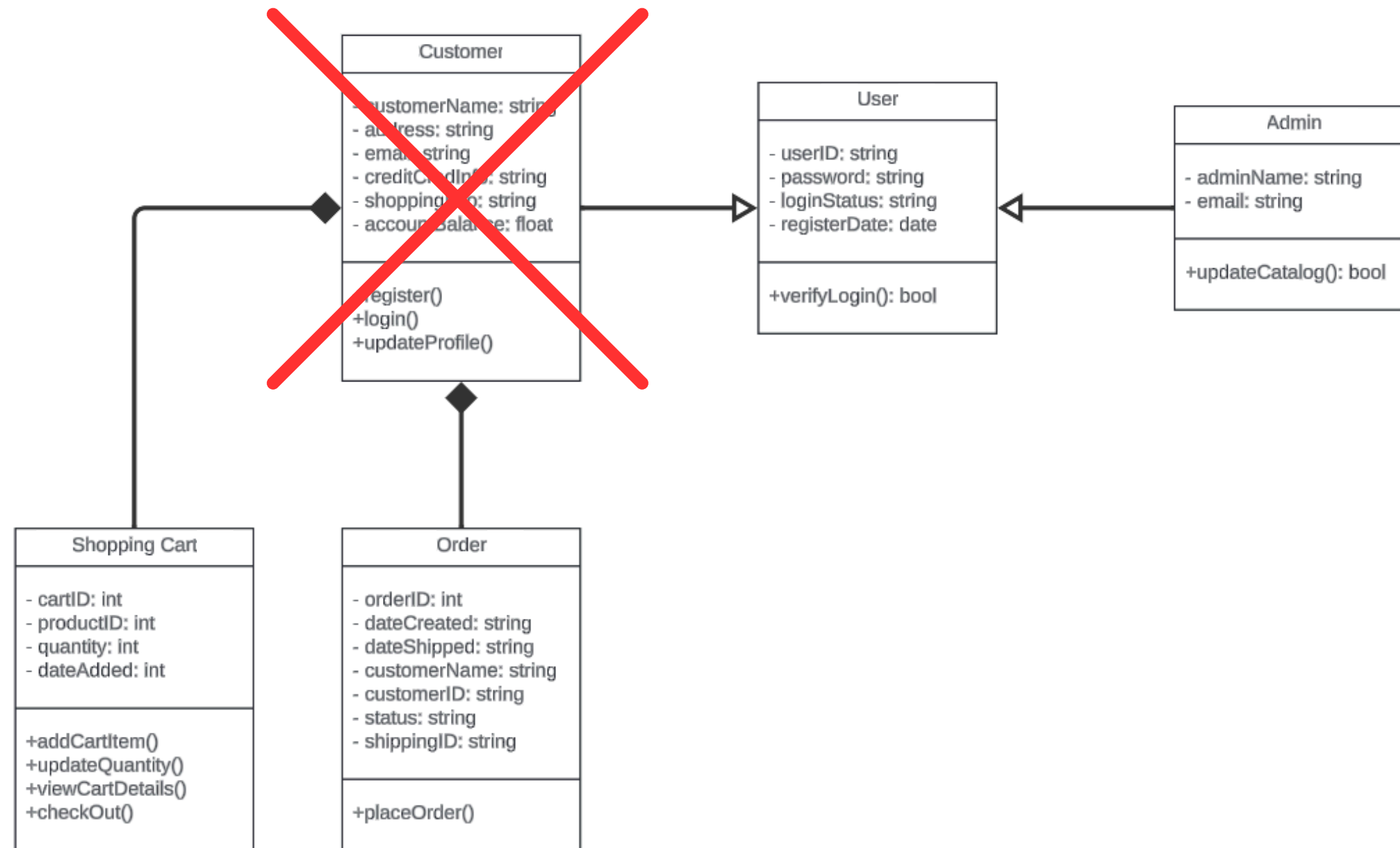


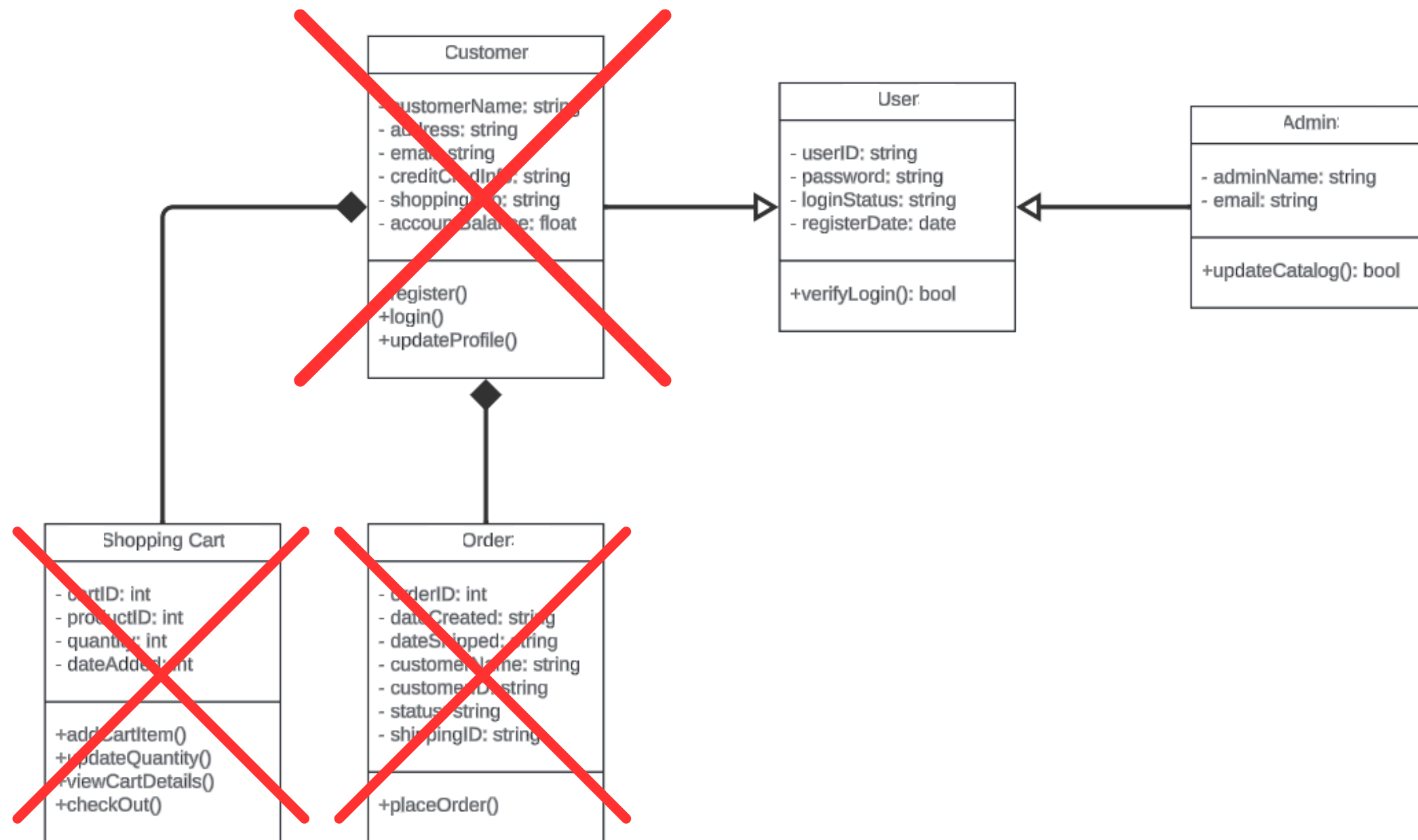


relationships
composition

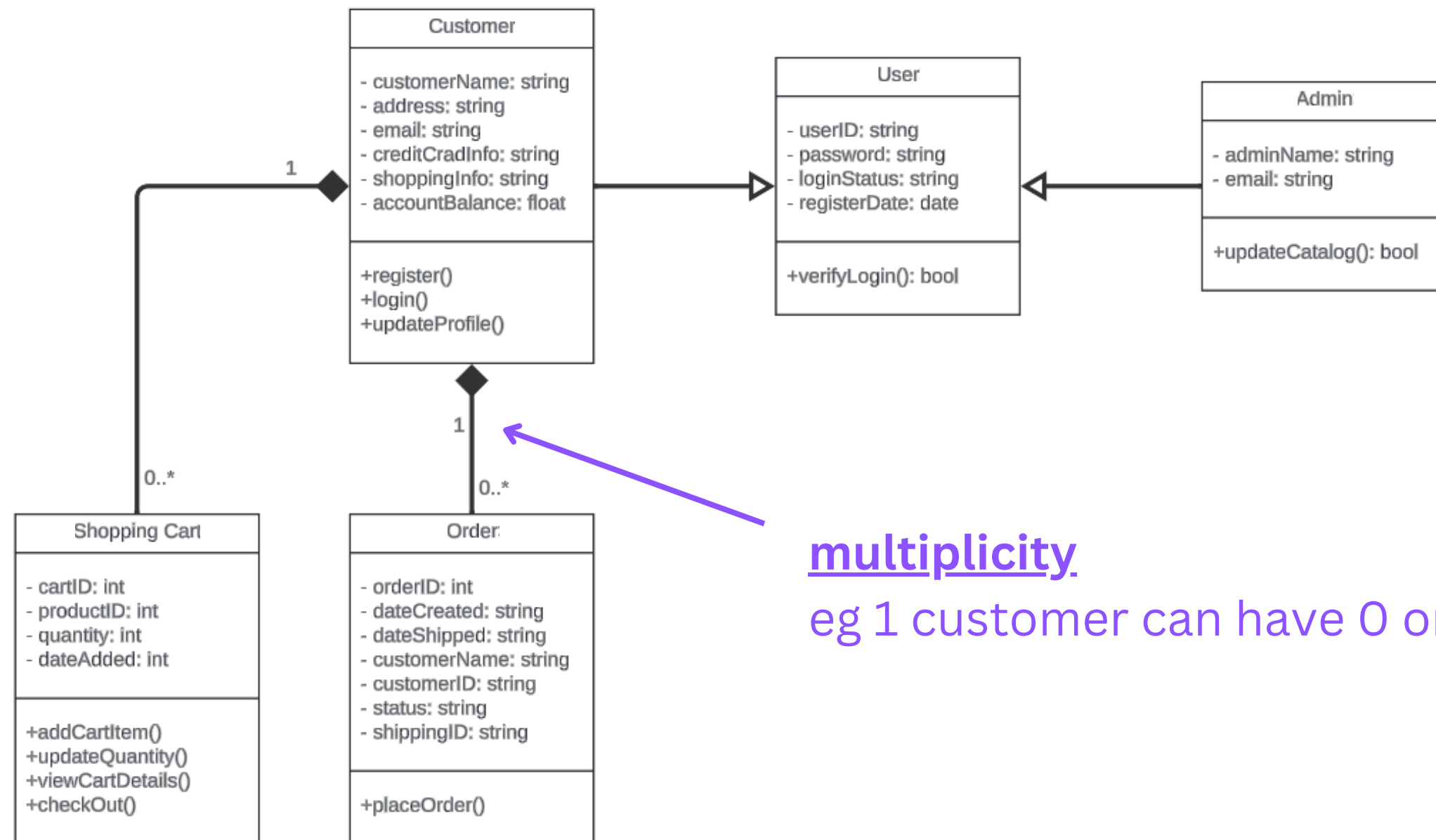


if the customer account is destroyed



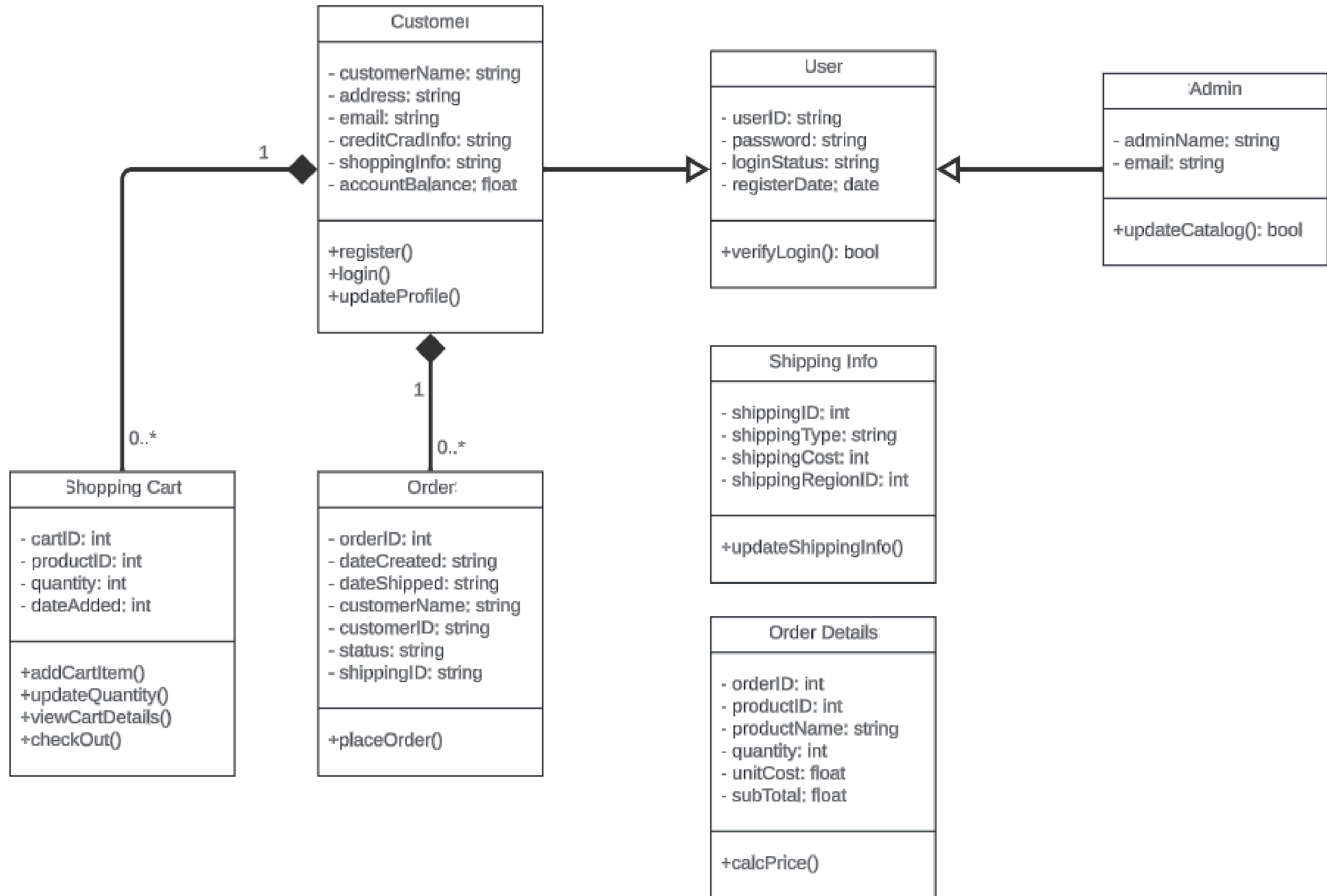


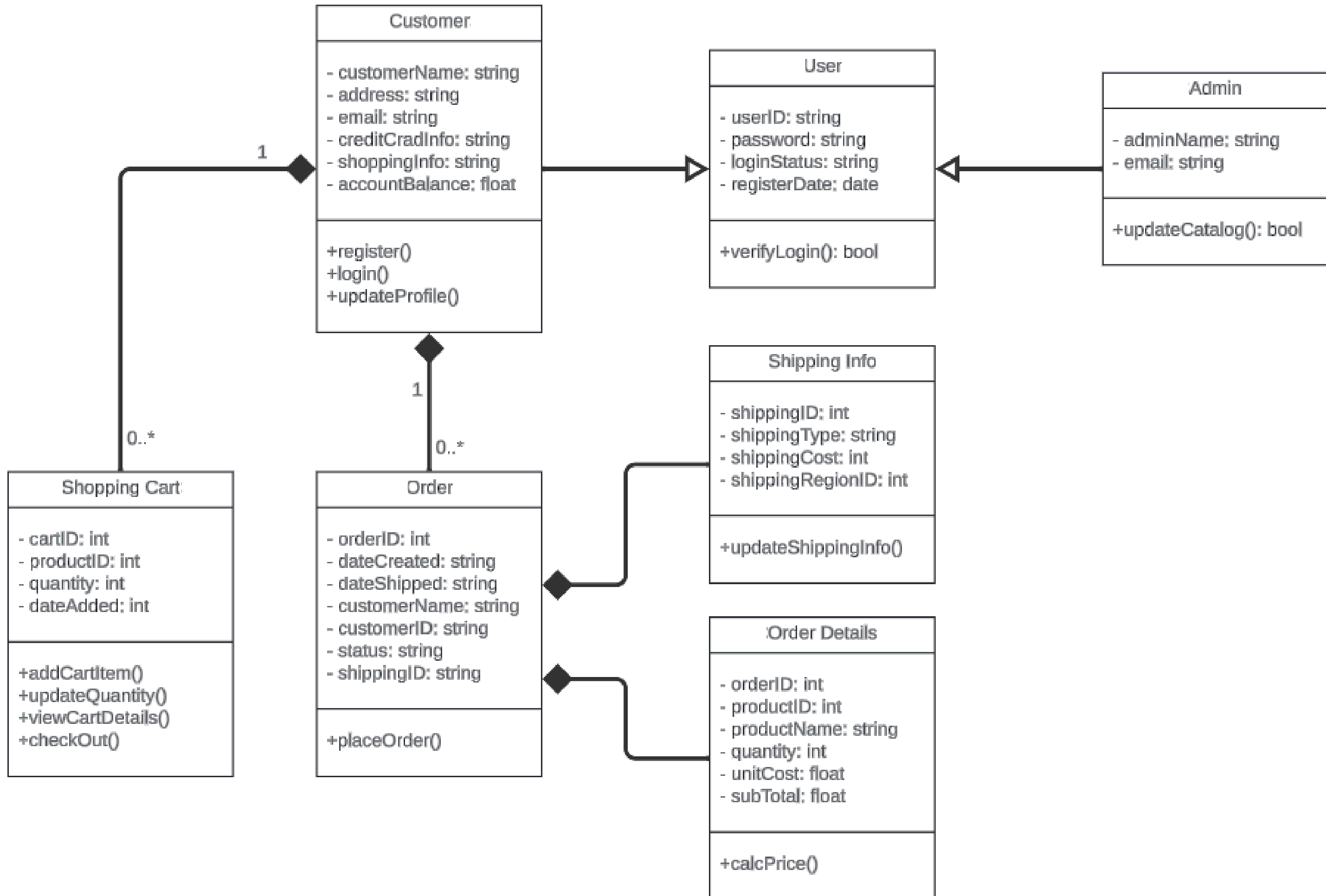
- shopping cart would be destroyed
- orders will be lost

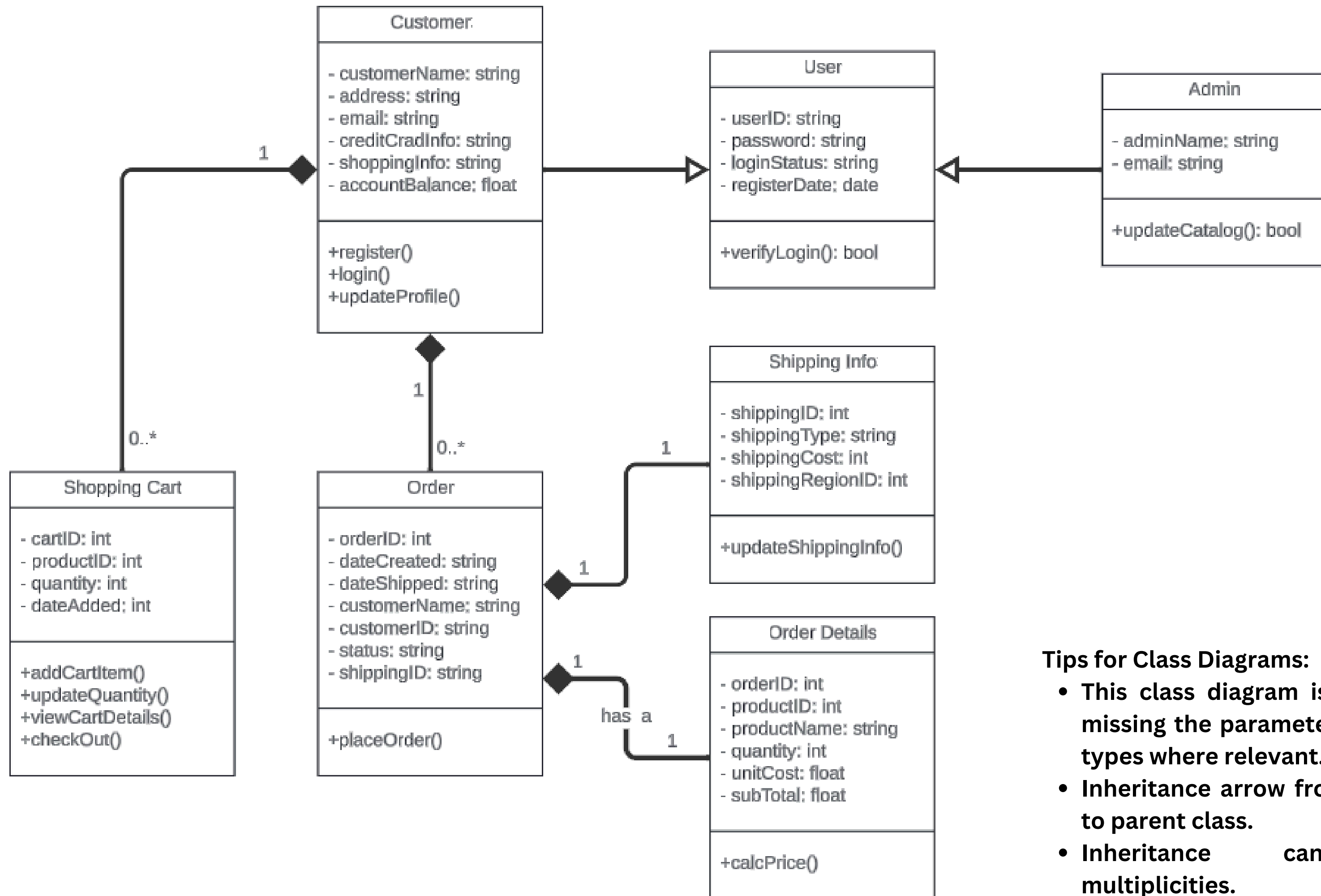


multiplicity

eg 1 customer can have 0 or ... many * orders







Tips for Class Diagrams:

- This class diagram is partial; it is missing the parameters and return types where relevant.
- Inheritance arrow from child class to parent class.
- Inheritance can include multiplicities.